



Université Tunis El Manar
École Nationale d'Ingénieurs
de Tunis



Department of Information and Communication Technologies

Second-Year Final Project Report

Prepared by :

Mariem Chaieb

Class :

2 TEL 2

AI-Based object detection application for blind people

Supervised by : **M. Hamza HAMMAMI**

2025 - 2026

ABSTRACT

For those who are blind or severely vision impaired simple tasks like crossing a busy street or identifying an object in a room represent a significant struggle .The traditional assistive tools have long been the gold standard for independence offering a limited information about the surroundings. Fortunately, we're entering a new era. Thanks to breakthroughs in DL and AI, we are moving beyond basic tools toward intelligent systems that can actually see, interpret and describe the world in RT replacing the role of the eyes for the people with such a struggle. This project presents the design and development of a mobile application "**BlindAid**" aimed at improving the independence and safety of blind and visually impaired individuals. The application uses the smartphone camera to continuously capture the user's surroundings, detects objects present in the scene using a DL model and instantly describes them to the user through audio output via a Text-to-Speech engine. Each detected object is announced along with its position relative to the user, providing clear and actionable navigation guidance.

The system was tested across multiple real-world environments and demonstrated reliable detection performance and low audio response latency, confirming its suitability for RT assistive use.

Keywords : mobile application, object detection, deep learning, Text-to-Speech, assistive technology, blind users, computer vision, accessibility, RT.

RÉSUMÉ

Les personnes aveugles ou sévèrement malvoyantes rencontrent des difficultés importantes dans la réalisation de tâches quotidiennes. Les outils d'assistance traditionnels sont bien qu'efficaces pour la mobilité de base en offrent une perception limitée et peu détaillée de l'environnement. Les avancées récentes en intelligence artificielle et en DL permettent désormais le développement de systèmes capables de percevoir, interpréter et décrire le monde en temps réel.

Ce projet présente la conception et le développement d'une application mobile "**BlindAid**" visant à améliorer l'autonomie et la sécurité des personnes aveugles et malvoyantes. L'application utilise la caméra du smartphone pour capturer en continu l'environnement de l'utilisateur ensuite elle détecte les objets présents à l'aide d'un modèle d'apprentissage profond, puis les décrit instantanément via une sortie audio grâce à un moteur de synthèse vocale (TTS).

Le système a été testé dans plusieurs environnements réels et a démontré des performances de détection fiables ainsi qu'une faible latence de réponse audio, confirmant son adéquation pour une utilisation assistive en temps réel.

Mots-clés : application mobile, détection d'objets, apprentissage profond, Text-to-Speech, technologie d'assistance, utilisateurs aveugles, vision par ordinateur, accessibilité, temps réel.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who contributed to the completion of this project through their support and encouragement.

I would especially like to thank my supervisor, Mr. Hamza Hammemi, for his guidance, availability and valuable feedback throughout this work. His support played an important role in shaping this project.

I also thank the members of the examination jury for taking the time to evaluate this work and for their valuable remarks and suggestions.

My thanks also go to the faculty members of the Department of Information and Communication Technologies at ENIT for the quality of education they have provided, which gave me the necessary knowledge to carry out this project. Finally, I wish to thank my family for their patience, their understanding and their unconditional support, without which this work would not have been possible.

LIST OF ACRONYMS

AGP	Android Gradle Plugin	RGB	Red Green Blue
AI	Artificial Intelligence	R-CNN	Region-based Convolutional Neural Network
API	Application Programming Interface	ReLU	Rectified Linear Unit
CNN	Convolutional Neural Network	RPN	Region Proposal Network
COCO	Common Objects in Context	RT	Real-Time
CUDA	Compute Unified Device Architecture	SDK	Software Development Kit
DL	Deep Learning	TFLite	TensorFlow Lite
GPU	Graphics Processing Unit	TTS	Text-to-Speech
IDE	Integrated Development Environment	UML	Unified Modeling Language
mAP	Mean Average Precision	VRAM	Video Random Access Memory
NMS	Non-Maximum Suppression	YOLO	You Only Look Once

CONTENTS

Abstract	ii
Résumé	iii
Acknowledgements	iv
List of Acronyms	v
General Introduction	1
1 Visual Accessibility and Computer Vision	3
1 Visual Accessibility and Assistance for the Visually Impaired	3
1.1 Mobility and Orientation Needs	4
1.2 Existing assistive technologies and their limitations	4
1.3 Contribution of AI to Accessibility	5
2 Foundations of Computer Vision	6
2.1 Overview	6
2.2 Object detection	7

3	DL for Object Recognition	7
3.1	Convolutional Neural Networks (CNNs)	8
3.2	Object Detection Models	10
3.3	RT constraints on mobile latforms	10
4	Conclusion	11
2	Object Detection Techniques and Vocal Description	12
1	RT image preprocessing	12
1.1	Digital pimage processing	12
1.2	Frame capture strategys	13
1.3	Resizing and normalization	14
1.4	Noise reduction	14
1.5	Color space conversion	15
2	Object detection techniques	16
2.1	Detection by CNN	16
2.2	YOLO models	18
2.3	Fine-Tuning YOLOv8	18
2.4	MobileNet and lightweight models	19
3	Object localization and prioritization	19
3.1	Bounding boxes	19
3.2	Position estimation	20
3.3	Filtering of critical objects	20
4	Vocal description	21
4.1	Text to speech conversion	21
4.2	TTS engines	22
4.3	Audio latency optimization	23
5	Conclusion	23

3	System Analysis and Design	24
1	Introduction	25
2	Requirements analysis	25
2.1	Target user profile	25
2.2	Use cases	26
2.2.1	Use case 1: Start RT detection	26
2.2.2	Use case 2: Receive audio guidance	26
2.2.3	Use case 3: Priority alert for critical objects	27
2.2.4	Use case 4: Stop detection	28
2.3	Mobility scenarios	28
3	System specifications	29
3.1	Functional requirements	29
3.2	Non-functional requirements	29
4	System architecture	30
5	UML modeling	31
5.1	Use case diagram	31
5.2	Class diagram	32
5.3	Sequence diagram	33
5.3.1	Sequence diagram 1 — Start Detection	33
5.3.2	Sequence diagram 2 — Receive Audio Description	33
5.3.3	Sequence diagram 3 — Stop detection	34
6	Conclusion	35

4	Implementation, Testing and Evaluation	36
1	Introduction	36
2	Development environment	36
2.1	Mobile platform	36
2.2	AI frameworks	37
2.3	Testing tools	38
3	Implementation of the Detection Module	39
3.1	Overview	39
3.2	Camera integration	39
3.3	Model loading and fine-Tuning	40
3.3.1	Dataset preparation	40
3.3.2	Training configuration	40
3.3.3	Fine-Tuning results	41
3.4	TFLite inference engine	42
3.5	RT post-processing	43
4	Implementation of the Vocal Description Module	43
4.1	Text generation	43
4.2	Speech synthesis	43
4.3	Audio queue management	44
5	Accessible User Interface	44
5.1	Simplified interface	44
5.2	Volume button control	45
5.3	Threading and lifecycle management	45
6	Testing and evaluation	45

6.1	RT performance	45
6.2	Real-world testing	46
6.3	Limitations	46
6.4	Future improvements	47
7	Conclusion	47
	General Conclusion and Perspectives	48
	Bibliographie	50

LIST OF FIGURES

1.1	General computer vision pipeline from image acquisition to interpretation .	6
1.2	DL-based object detection pipeline	8
1.3	CNN architecture illustrating the hierarchical transition from raw input pixels to high-level feature descriptors.	9
2.1	Image preprocessing pipeline for object detection	13
2.2	Comparison of image enhancement techniques applied to the cameraman grayscale image by a Gaussian filtering and median filtering on Matlab . .	15
2.3	System architecture diagram for the computer vision assistive application.	16
2.4	Faster R-CNN architecture [7]	17
2.5	Rule-based natural language generation pipeline	21
2.6	Two-stage TTS synthesis process	22
3.1	On-device processing pipeline	30
3.2	Use case diagram for the object detection system established using StarUML	31
3.3	Class diagram of the Object Detection Assistance System using StarUML .	32
3.4	Sequence diagram: Start detection	33

3.5	Sequence diagram: Receive audio description	34
3.6	Sequence diagram: Stop detection	35
4.1	TensorFlow Lite framework used for on-device inference (source: [?])	37
4.2	BlindAid's processing pipeline.	39
4.3	Roboflow dataset configuration	40
4.4	YOLOv8 Fine-Tuning configuration on Google Collab	41
4.5	YOLOv8 fine-tuning performance metrics	42
4.6	BlindAid logo.	44
4.7	BlindAid main interface detecting Bsbs with 84% accuracy	44

LIST OF TABLES

2.1	Object priority classification for audio filtering	20
3.1	Use case description: Start RT Detection	26
3.2	Use case description: Receive audio guidance	27
3.3	Use case description: Priority alert for critical objects	27
3.4	Use case description: Stop detection	28
4.1	Mobile Development Environment	37
4.2	AI Development Environment	38
4.3	YOLOv8 Fine-Tuning performance Metrics	41
4.4	Position Estimation Logic	43

GENERAL INTRODUCTION

The growing capabilities of artificial intelligence and mobile technology are opening new possibilities for people who were previously under-preserved by existing tools. Among the populations that stand to benefit most from these advances are individuals with visual disabilities.

According to the World Health Organization approximately 2.2 billion people worldwide live with some form of vision impairment of whom at least 1 billion have a condition that could have been prevented or has yet to be addressed [1]. This number alone highlights the urgency of developing accessible and affordable assistive solutions.

For blind or a visually impaired person navigating everyday environments presents daily challenges. Identifying objects or avoiding obstacles, which a fully sighted individuals perform automatically and effortlessly require a higher physical effort when the visual input is absent.

Traditional assistive tools provide only limited information about the surrounding environment as they can only detect that an obstacle exists but cannot identify what it is, where exactly it is or how urgently the user needs to respond to it. This gap between what users need and what current tools offer is the starting point for this project.

The primary objectives of this project are to enable visually impaired users to identify surrounding objects and understand their spatial position thereby improving their safety during navigation.

The system is designed to deliver information instantly through audio output, eliminating the need for any visual interaction with the device. It also prioritizes safety critical objects ensuring that the most important elements in the environment are communicated first. In addition, the application aims to provide a simple and accessible user experience that can be easily used without any prior technical knowledge or training.

This report is organized into four chapters, framed by this general introduction and a general conclusion:

- Chapter 1 – **Visual Accessibility and Computer Vision** Presenting the challenges of visual impairment, surveying existing assistive technologies and their limitations and introducing the theoretical foundations of computer vision and DL used in this project.
- Chapter 2 – **Object Detection Techniques and Vocal Description:** Details the complete technical approach by covering image preprocessing, the YOLOv8 detection pipeline with TFLite on-device inference, spatial localization and priority filtering logic and TTS synthesis.
- Chapter 3 – **System Design:** Presenting the requirements analysis, functional and non-functional specifications, system architecture and UML modeling
- Chapter 4 – **Implementation, Testing and Evaluation:** Describing the development environment, walking through the implementation of all system components, showing testing methodology and results and discusses limitations and future improvements.

CHAPTER 1

VISUAL ACCESSIBILITY AND COMPUTER VISION

Introduction

This chapter presents the context and theoretical foundations of the project. It covers three main areas: the real challenges faced by blind and visually impaired individuals in daily navigation, a critical review of existing assistive technologies and their limitations and the core concepts of computer vision and DL on which the proposed system is built upon.

1 Visual Accessibility and Assistance for the Visually Impaired

Visual impairment refers to a the loss of vision that remains uncorrected by eyeglasses, medication or surgical operations .It is classified into two main categories: **low vision**, where the person retains partial sight but with functional limitations and **complete**

blindness defined by the complete or near-complete absence of visual perception. Beyond its medical definition, visual impairment affects daily life as it disrupts a person’s ability to navigate physical spaces safely and to interact with the surrounding environment. Understanding these challenges is essential before proposing any technological solution.

1.1 Mobility and Orientation Needs

Visual perception is the primary sensory input for spatial orientation and environmental awareness. When it is absent, any simple tasks become a challenge that necessitate additional techniques, assistive technologies or support from others.

The mobility difficulties fall into three main domains. First **obstacle detection and avoidance** which involve the identification of both hazards like furniture and stairs and risks such as vehicles or moving objects. Those obstacles need the most critical safety requirement. Second, **object identification and interaction** moving beyond detection toward recognizing objects by category and position to perform routine tasks.

Finally, **environmental orientation and navigation** which is the ability to understand the spatial layout of an environment, to identify landmarks or to plan safe routes through unfamiliar areas.

These challenges imply that an effective assistive system should prioritize low latency, provide precise spatial context and rely on a fully non-visual interface all within a portable and affordable solution suitable for all users.

1.2 Existing assistive technologies and their limitations

Several categories of assistive technology currently serve visually impaired users.

The white cane remains the most widely used tool by providing reliable physical feedback for ground-level obstacles but it still lacks the ability to offer precise information about the nature of those objects.

A more advanced wearable solutions such as the OrCam MyEye [2] which is a wearable device that clips onto one’s eyeglasses and uses a built-in camera to read text, recognize faces, identify objects and deliver results through a small earpiece speaker. While effective,

this device carry a cost that places them out of reach for the majority of visually impaired users.

Furthermore, mobile platforms like Be My Eyes[3] or Aira [4] facilitate RT assistance by connecting users with sighted volunteers or remote AI. The problem is that these two services depends on high-speed internet connectivity and may not provide the rapid response required for daily navigation assistance .

These limitations highlight the need for a smarter approach where AI takes over the role that human sight can no longer fulfill while still affordable for most users .

1.3 Contribution of AI to Accessibility

Deep learning has transformed assistive technology by overcoming many limitations of traditional computer vision. Modern models learn representations directly from data which introduce higher performance in complex conditions. Among these advantages, three stands the most:

First, the models trained on large-scale datasets like COCO [6] can recognize a wide range of everyday objects especially when paired with the added flexibility of fine-tuning for specific domain improvements.

Although COCO provides a strong foundation by covering 80 common object classes [6] yet it is not specifically designed for assistive navigation. Several objects are under-represented or absent which may reduce reliability . This limitation motivates the fine-tuning approach where the pre-trained YOLOv8 model is further trained to improve detection accuracy.

Second, RT performance is achieved through single-stage detectors like YOLOv8 which enables fast processing on standard mobile hardware.

Third, AI offer robustness across variations in lighting, viewpoints and object scale that help ensuring consistent performance .

When combined with TTS systems and smartphones, those elements create an affordable foundation for an intelligent assistive system.

2 Foundations of Computer Vision

2.1 Overview

Computer vision is a field of AI that enables machines to interpret and understand visual information captured from different sources .

In the context of this project, computer vision is not treated as a research topic but more as a functional component of an assistive system. Its role is to continuously analyze the user’s surroundings through a smartphone camera and extract information that can be translated into spoken guidance.

A typical computer vision system follows a structured pipeline.

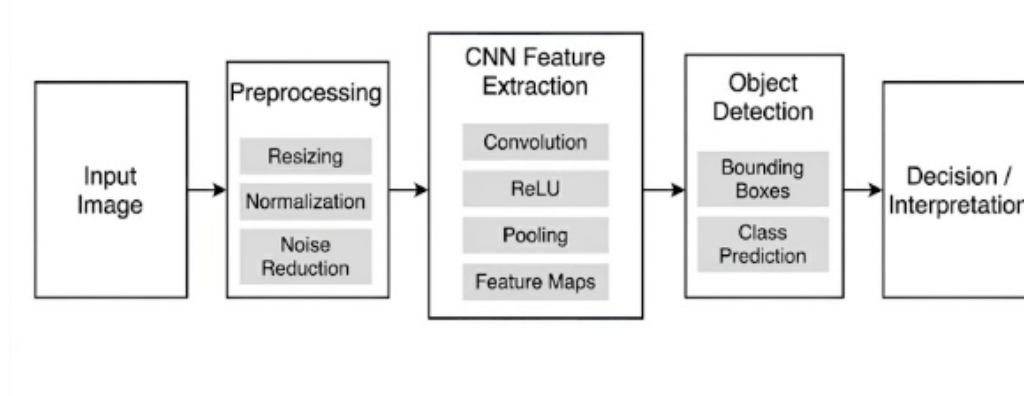


Figure 1.1: General computer vision pipeline from image acquisition to interpretation

An image passes through a preprocessing stage where it is resized, normalized and converted into a suitable format for the model. Then processed by a feature extraction module that is based on Convolutional Neural Networks (CNNs). At a technical level, computer vision systems rely heavily on CNNs to process visual input and to identify important visual patterns such as edges, textures and shapes. These models learn directly from data enabling them to adapt to complex and dynamic environments. A more detailed explanation of CNN-based feature extraction is provided in section 3.1. Finally, the extracted features are used in the interpretation stage to detect and locate objects in the scene.

Among the different computer vision elements , **object detection** is the most used for assistive systems . Unlike image classification which only identifies what is present in

an image, object detection provides both the identifying and the spatial location of an objects . The following sections will detail how these concepts are applied.

2.2 Object detection

Object detection is a specific computer vision task that combines **image classification** which identifies the type of an object with **localization** which determines its position within an image.

The localization is expressed using a bounding box drawn around each detected object. Each bounding box is associated with a class label indicating the object category, as well as a confidence score that reflects how certain the model is about its prediction.

In contrast, classification is the ability of a model to assign a label to a given input. An image classification model is trained to recognize visual patterns and map them to a set of known categories . This is achieved by learning discriminative features from large collections of labeled images during a training phase. The stronger and more diverse the training data, the better the model generalizes to new unseen images.

This combination of classification and localization is what makes object detection suitable for real-world applications. In this project, the spatial information obtained from object detection is used to determine the relative position of objects and to generate appropriate audio feedback

Modern object detection systems are based on DL techniques that allow models to automatically learn relevant visual features from data, leading to improved accuracy and robustness in complex environments.

3 DL for Object Recognition

Deep learning has improved the performance of visual aid systems by enabling models to automatically learn visual features directly from data. The detection relies on CNNs that are capable of recognizing complex patterns and structures from large-scale datasets leading to high accuracy in real-world scenarios.

In a DL based object detection system, the process follows a structured pipeline that transforms raw visual data into information.

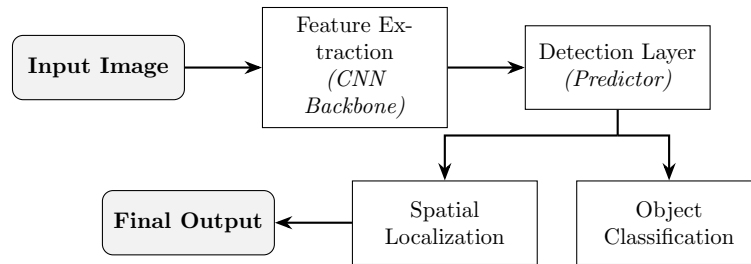


Figure 1.2: DL-based object detection pipeline

As illustrated in 1.2: an image is captured from the camera and passed to the model as input. The CNNs process it through feature extraction and detection layers to produce structured predictions in the form of object classifications and spatial localization results. The final output consists of labeled objects along with their spatial locations in the image. This pipeline serves as the structural backbone of the application. By utilizing DL, the architecture maintains consistent detection accuracy across unpredictable environments and generates helpful audio feedback that allows the system to function as a reliable navigational aid.

3.1 Convolutional Neural Networks (CNNs)

CNNs are a class of DL architectures specialised for processing data such as images. Unlike traditional deep learning models that require manual feature engineering, CNNs utilize a hierarchical structure to learn spatial features ranging from primitive edges to complex semantic object patterns directly from raw pixel data.

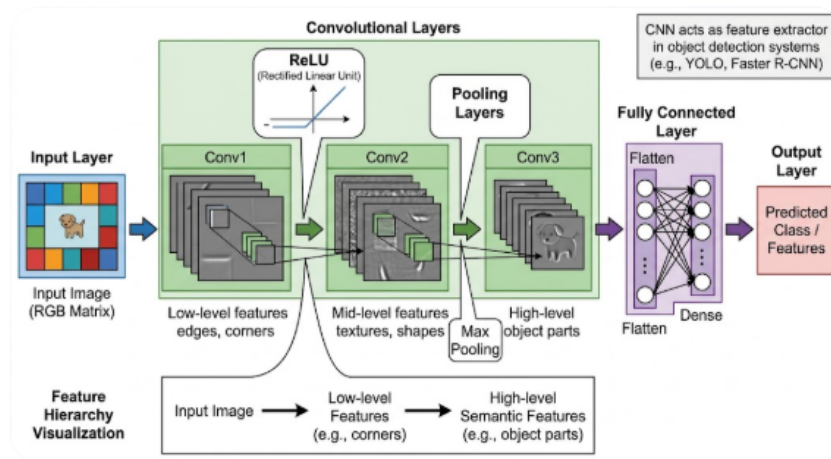


Figure 1.3: CNN architecture illustrating the hierarchical transition from raw input pixels to high-level feature descriptors.

The standard pipeline, as shown in figure 1.3, consists of a sequence of functional layers:

- **Convolutional Layers (Conv1- Conv2- conv3):** those layers apply spatial filters to perform local feature extraction. Initial layers capture low-level details like edges and corners and deeper layers analyses these into mid-level textures and high-level object components.
- **Activation Function:** a Rectified Linear Unit (ReLU) is applied to introduce non-linearity by that enabling the network to learn complex mappings beyond simple linear transformations.
- **Pooling Layers:** performs downsampling to reduce the spatial dimensionality of the feature maps. This process decreases computational load and provides a degree of translation invariance.
- **Fully Connected Layers:** after the final feature extraction, the multi-dimensional maps are flattened into a 1D vector. These layers act as a classifier or regressor ensuring the abstract descriptors to produce final detections.

CNNs function as the backbone architecture as they are responsible for generating the feature representations that high-level detection heads require to achieve precise identification and localization in RT.

3.2 Object Detection Models

Building on the feature extraction capabilities of CNNs, object detection models extend this functionality by transforming extracted feature maps into structured predictions. While CNNs are responsible for learning visual representations such as previously mentioned (edges- textures-shapes), object detection models exploit these representations to simultaneously localize and classify multiple objects in an image .

Modern object detection approaches can be broadly divided into two categories: **two-stage detectors and single-stage detectors**.

Two-stage detectors first generate candidate object regions and then classify them. Those models achieve high accuracy yet they're computationally expensive and not suitable for RT applications. In contrast, single-stage detectors perform detection and classification in a single pass through the network. This approach improves processing and speed . The YOLO family is among these models.

In this project, the YOLO model was selected as the detection engine. Its architecture builds directly upon CNN-based feature extraction and followed by optimized prediction layers that efficiently generate bounding boxes and class labels in a single forward pass.

YOLO uses the richness of CNN feature maps to produce dense and immediate predictions across the entire image.

To further adapt the model , the pre-trained network was fine-tuned on a custom COCO dataset. This what enhances the accuracy of recognized objects and the contextual relevance.

Finally, a lightweight configuration was used to ensure efficient execution on mobile hardware. This what allows continuous RT detection with minimal dela.

3.3 RT constraints on mobile latforms

Deploying an object detection system on a mobile device introduces several practical constraints that directly impact performance and design choices. Unlike desktop environments, smartphones operate with limited computational resources, restricted memory and strict energy constraints.

These limitations affect the feasibility of running DL models locally at higher frame rates. Memory constraints restrict the deployment of large models leading to reduced performance. Another important factor is latency which is that the time between capturing an image and generating audio feedback must stay minimal to ensure that the information is still relevant for navigation.

To address these constraints, this project adopts a fully on-device architecture. Rather than off-loading computation to an external server, the detection model was exported to TensorFlow Lite format and deployed directly on the device. This design choice eliminates any dependency on network connectivity or external infrastructure ensuring that the application remains fully functional in all environments with no internet access which is essential for a reliable assistive application.

4 Conclusion

This chapter presented the context and main concepts needed for this project as it described the challenges faced by visually impaired users in daily navigation and reviewed the limitations of existing assistive solutions. It has also introduced the main ideas of computer vision and DL used in modern object detection systems including CNN-based feature extraction and RT detection models such as YOLO. Finally it discussed the constraints of mobile platforms that influenced the design choices of the proposed system.

The next chapter will focus on object detection techniques and the vocal description system used in the application.

CHAPTER 2

OBJECT DETECTION TECHNIQUES AND VOCAL DESCRIPTION

Introduction

Building on the theoretical foundations presented in Chapter 1, this chapter focuses on the practical implementation of the proposed system. Each stage is examined from the moment a camera frame is captured to the moment the user hears a spoken description of their surroundings.

1 RT image preprocessing

1.1 Digital pmage processing

A digital image is a two dimensional array of pixels where each pixel stores color information typically encoded in three channels RGB each with values in the range $[0, 255]$. For DL models, raw pixel values are normalized to facilitate gradient based optimization.

Before a visual scene can be understood by a DL model, the raw pixel data must go through a series of preprocessing steps that prepare it for analysis. In fact, any error at this stage directly affect the precision of everything that follows. These preprocessing steps are important for the accuracy of object detection systems although modern DL models are reboust enough .

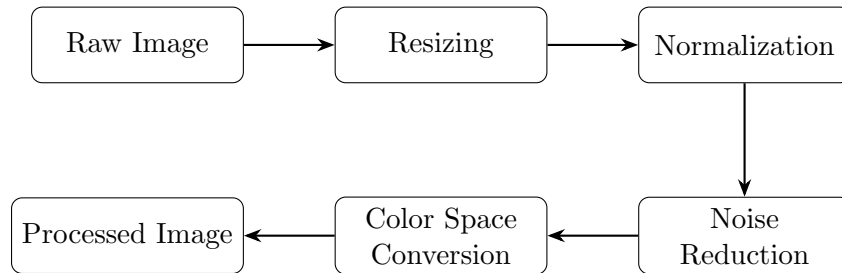


Figure 2.1: Image preprocessing pipeline for object detection

The following subsections discuss each stage of the pipeline [2.1](#) in detail.

1.2 Frame capture strategys

The first step in any visual detection pipeline is the acquisition of image data from the real world.

In the context of mobile assistive applications, this means capturing frames from the device’s camera in real time. Images can be acquired as individual frames or as a continuous video stream. For RT assistive applications, a frame by frame approach is preferred over full video streaming because it allows the system to control the processing frequency, manage network bandwidth efficiently and adapt the detection rate to the available computational resources.

The capture frequency impacts as well the responsiveness of a system. Too low of a frequency makes the system slow to react to environmental changes and too high one overloads the processing pipeline and wastes energy. In our project, frames are captured at regular intervals from the smartphone camera and transmitted to the detection model, providing a continuous flow of visual information.

1.3 Resizing and normalization

Raw images captured from cameras come in a variety of resolutions and formats that are not usually compatible with the input requirements of DL models. Before an image can be processed by a detection network it must undergo a standardization process to ensure consistency across all inputs. This process involves resizing and normalization.

Resizing adjusts images to the dimensions expected by the model, typically 640×640 pixels in the case of YOLOv8. Images that do not match this size are resized accordingly with letterboxing to preserve the original aspect ratio and avoid geometric distortion of the objects within the scene.

Normalization on the other hand, converts raw pixel values which range from 0 to 255 as integers into floating point values within a smaller, standardized range, typically between 0.0 and 1.0. This rescaling improves numerical stability and ensures consistent model behavior during inference.

Although simple, this steps ensures that the model receives inputs that matches the conditions under which it was trained and by that its predictions are therefore accurate.

1.4 Noise reduction

Images are often affected by factors such as low lighting conditions, motion blur, sensor limitations and compression artifacts all of which can introduce noise and degrade visual quality. Such distortions impact negatively the performance by obscuring irrelevant features. For this reason, noise reduction techniques are considered as a preprocessing step and their objective is to suppress unwanted image variations while preserving meaningful structures.

Among classical denoising methods, Gaussian filtering smooths the image by averaging pixel values within a local neighborhood, while median filtering replaces each pixel with the median value of surrounding pixels, which make it effective against impulse noise while better preserving edges.

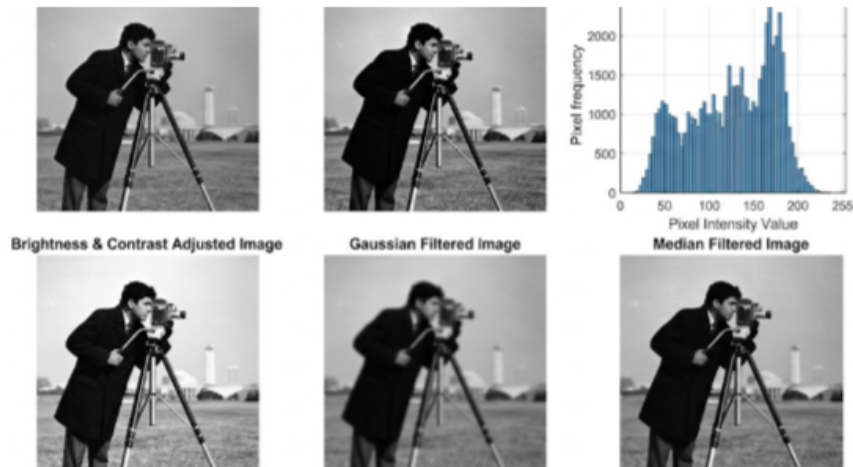


Figure 2.2: Comparison of image enhancement techniques applied to the cameraman grayscale image by a Gaussian filtering and median filtering on Matlab

To illustrate better the effect of these techniques, a MATLAB experiment was conducted using a grayscale test image, shown in Figure 2.2. The gaussian filtering produces global smoothing whereas median filtering suppresses noise more selectively while preserving structural details. The brightness and contrast adjustment example further highlights how illumination enhancement can interact with perceived noise.

Although those preprocessing methods can improve image quality, models like YOLOv8 are trained on datasets containing images with varying levels of noise and quality degradation providing robustness to moderate noise. Consequently, the denoising was studied as a preprocessing consideration but was not adopted as a necessary stage in the final pipeline.

1.5 Color space conversion

Color Space Conversion is the process of transforming images from one color representation to another formats depending on the application. In DL based vision systems, this step is used to simplify the input data or to enhance specific visual features which helps on reducing computational complexity while still preserving essential structural information.

2 Object detection techniques

Having understood how raw images are prepared for processing, moving forward to exploring the techniques of the detection pipeline. Before diving into each technique in detail, the figure below provides an overview of the complete pipeline illustrating how all the stages described in this chapter connect together.

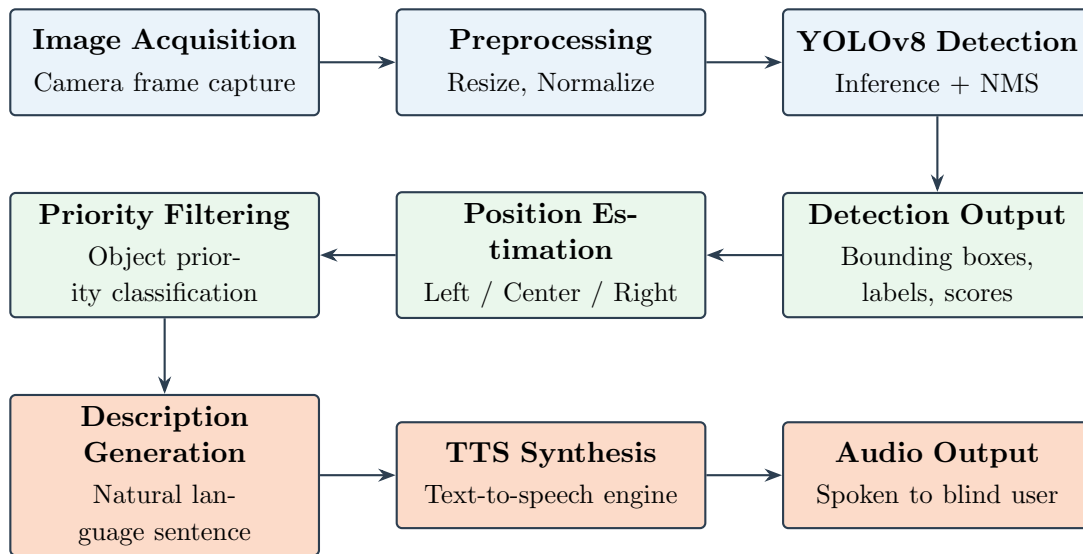


Figure 2.3: System architecture diagram for the computer vision assistive application.

The following sections describe each detection approach of this pipeline in detail.

2.1 Detection by CNN

As introduced in Chapter 1 section 3.1, CNNs learn to extract visual features from raw pixel data through successive layers of convolution, activation and pooling.

In a classification CNN, the spatial feature maps produced by the convolutional layers are eventually flattened and passed through fully connected layers that collapse all spatial information into a single output vector answering only the question *"what is in this image"*. A detection CNN must preserve spatial information throughout the network in order to also answer *"where is it?"*. This is achieved by keeping the convolutional feature maps intact and attaching specialized prediction heads that interpret both the semantic content and the spatial coordinates encoded in those maps.

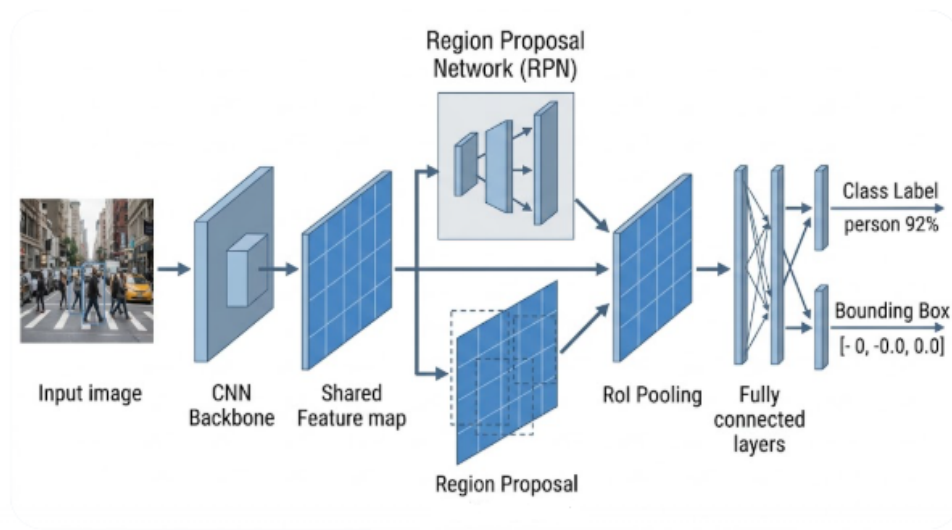


Figure 2.4: Faster R-CNN architecture [7]

The first generation of CNN-based detectors, **R-CNN** [7] operated in two sequential stages. As shown in Figure 2.4, the input image is first passed through a CNN backbone to produce a shared feature map. A selective search algorithm then scans this map to generate around 2,000 candidate region proposals each of which is individually warped and passed through the CNN for classification.

While this demonstrated that CNNs could be highly effective for detection, the sequential per-region processing made it slow requiring up to 46s for a single image making the detection unsuitable for RT assistive use.

Faster R-CNN addressed this by introducing the Region Proposal Network (RPN) which replaced the external selective search with a learned sub-network operating directly on the shared feature map. The candidate regions it produces are then extracted via Pooling and passed through fully connected layers that simultaneously output both a class label and precise bounding box coordinates.

Despite these improvements, two-stage architectures retain a sequential dependency where region proposals must be generated before classification can begin. Such limitation directly motivated the development of single-stage detectors, "the YOLO family" which resolves detection and classification simultaneously in a single forward pass.

2.2 YOLO models

Among single-stage object detection architectures, the YOLO family has become a reference for RT application by dividing the input image into a grid and simultaneously predicting bounding boxes, objectness scores and class probabilities for all cells at once. This architectural decision is what makes it perform detection in a single forward pass. The latest version, YOLOv8 [5] selected as the detection model of this project represents the most refined iteration of the family to date. Several of its architectural characteristics make it particularly well suited to the constraints of this application.

Unlike earlier approaches, YOLOv8 adopts an anchor-free detection strategy by directly predicting object locations without relying on predefined anchor boxes. This simplifies the detection process across objects with varying shapes and sizes. In addition, the model introduces a decoupled detection head which separates localization and classification tasks into different branches enhancing prediction and accuracy by allowing each task to be optimized independently.

At the architectural level, YOLOv8 contains improved feature extraction and a fusion mechanisms. The model also incorporates C2f modules that enhance feature propagation and re-use resulting in richer representations with lower computational cost. These features are then processed through a PAN-FPN-based neck which combines multi-scale information to ensure robust detection of both small and large objects in complex environments.

2.3 Fine-Tuning YOLOv8

Although YOLOv8 pre-trained on the COCO dataset provides strong general detection capabilities, it still lacks coverage of important objects related to navigation. To address this gap, the model was fine-tuned using a COCO-based custom dataset of 10,164 annotated images sourced from Roboflow [10] and integrated via its API into a Google Colab environment [15]. This approach aims to retain previously learned visual features while adapting the model to the target domain using the standard YOLO loss function and data augmentation techniques (scaling, flipping, brightness variation) to improve robustness.

The annotation quality and class selection were carefully aligned with navigation requirements to ensure that the model focuses on objects that are related to safety and mobility.

Overall, fine-tuning allows the model to achieve better understanding of the environment while maintaining efficient inference suitable for RT assistive applications.

2.4 MobileNet and lightweight models

The current on-device architecture runs the entire inference pipeline directly on the smartphone using a TFLite model . This already achieves fully offline operation without any server-side processing or network dependency. A natural direction for future development would be exploring even lighter model variants such as MobileNet that could further reduce inference time and memory footprint, making the application viable on a wider range of entry-level devices while maintaining acceptable detection accuracy

3 Object localization and prioritization

Detection alone is not sufficient for an effective assistive system. Knowing that an object exists provides limited navigational value if the user cannot determine the position of the object relative to their own position. Object localization and prioritization extend beyond the raw detection output by transforming a list of labels into actionable guidance .

3.1 Bounding boxes

The model produces a bounding box for each detected object defined by four coordinates (corners) that tightly encloses the object within the image frame. based on these coordinates , the system determine the center point and the box dimensions within the frame. The center is used to estimate the object’s location while the box size provides an approximation of its distance.

3.2 Position estimation

Once the bounding box center has been detected, the next step is to translate its position within the image frame into a directional description that corresponds to the user’s natural spatial reference frame. This is achieved by dividing the image width into three horizontal zones of equal size. An object whose normalized horizontal center falls below 0.33 is announced as being to the left, between 0.33 and 0.66 as directly ahead and above 0.66 as to the right. Vertical elements produce additional information. Objects occupying a large bounding box are classified as nearby while smaller boxes indicate greater distance. Three levels are used (nearby-ahead-in the distance) each conveying a different degree of urgency to the user.

By combining horizontal and vertical position analysis, a detailed descriptions such as ”person ahead in the distance” or ”chair to the left, very close” delivers precise information to the user.

3.3 Filtering of critical objects

Beyond estimating where objects are, the system must also determine which objects are worth announcing and in what order.

The system addresses this through three complementary mechanisms.

Table 2.1: Object priority classification for audio filtering

Priority Level	Object Categories	Behavior
High	Person, vehicle, traffic lights, stairs, doors	Always announced first, included at slightly lower confidence thresholds
Medium	Chairs, tables, benches, sofas, furniture	Announced after all high-priority detections at standard confidence threshold
Low	Decorative items, background objects, small household items	Only announced when no higher priority objects are present

As summarized in table 2.1, each priority level directly determines both the order and the conditions under which objects are announced.

Beyond priority, a maximum detection count per announcement cycle is enforced selecting only the top detections ordered by priority level and by confidence score. This ensures that each announcement remains heard and understood before the next detection cycle begins.

Together these mechanisms ensure that every announcement delivered to the user is useful for a safe navigation.

4 Vocal description

Once objects have been detected, localized and filtered according to their priority level, the final stage of the pipeline transforms the detection results into spoken language that the user can immediately act upon. Text-to-Speech synthesis is what makes the entire system accessible to a blind user.

4.1 Text to speech conversion

The first step in the vocal output pipeline is constructing a natural language description from the structured detection results. Each detected object is described using a compact phrase combining its class label, directional position and distance estimate.

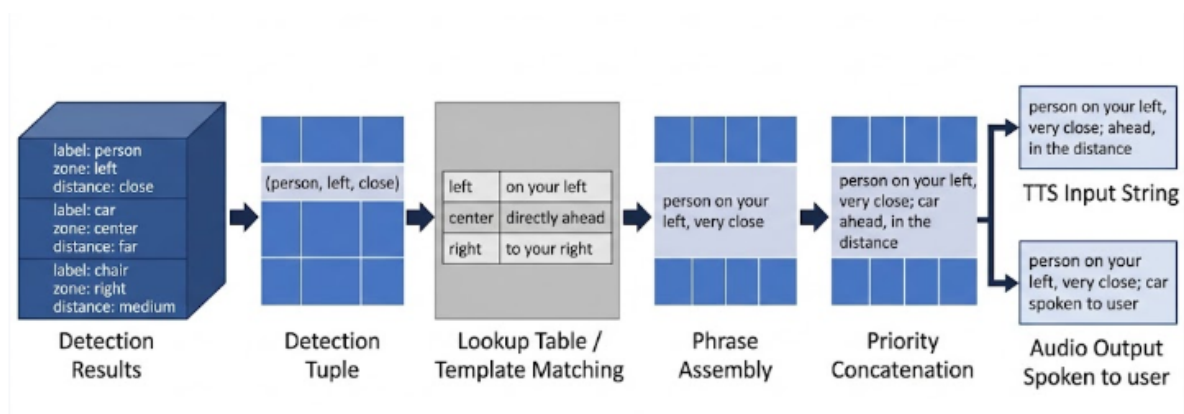


Figure 2.5: Rule-based natural language generation pipeline

As illustrated in the figure 2.5, the conversion is made possible through a rule based natural language generation module that maps structured detection data directly to text templates. Each detection tuple that contains a class label, a zone index and a distance

level is passed through a lookup table that selects the appropriate positional qualifier and assembles the final phrase. **No neural language model is involved at this stage** This approach is considered as simple since it guarantee low-latency output regardless of the number of detected objects.* The assembled phrases are then concatenated in priority order and passed as a single string to the TTS engine.

With the natural language description assembled, it is passed to the TTS engine which converts it into the spoken audio output the user ultimately hears.

4.2 TTS engines

Text-to-Speech engines are the software components responsible for converting a text string into audible speech.

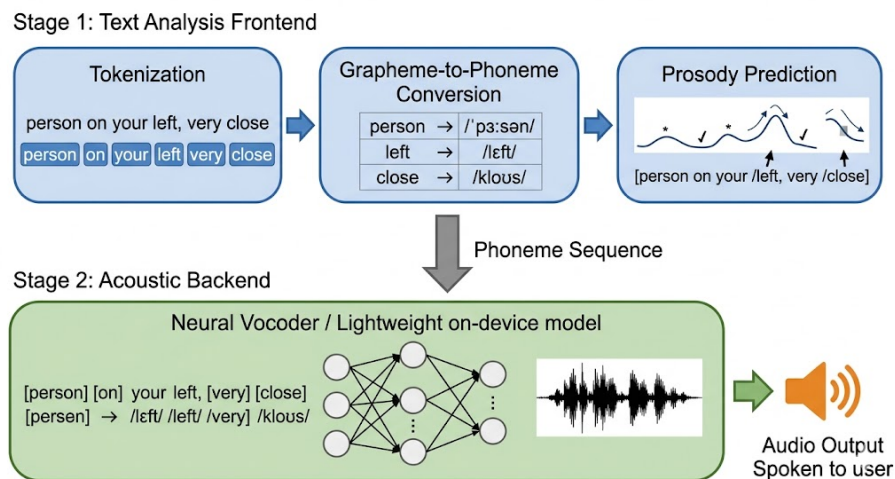


Figure 2.6: Two-stage TTS synthesis process

TTS engines operates through a two-staged process. As illustrated in Figure 2.6, the TTS synthesis process begins with a linguistic frontend that tokenizes the input string, maps each token to its phonemic representation and predicts the appropriate prosody before passing the resulting phoneme sequence to the on-device neural vocoder for waveform synthesis.

Unlike cloud-based solutions such as Google WaveNet and Tacotron 2 that depend on server-side inference and may introduce latency, the adopted approach operates on-device processing to ensure fast and reliable speech generation without network dependency.

Configurable parameters like speech rate, pitch and language allow personalization of the audio output. A deduplication mechanism prevents redundant announcements by suppressing repeated descriptions of unchanged objects.

All features combined make this design efficient with minimal latency providing RT audio feedback.

4.3 Audio latency optimization

Minimizing audio latency requires attention at every stage of the pipeline.

On the capture side by using compressed image formats reduces the volume of data that must be transmitted and by ensuring that only the most recent frame enters the inference pipeline, preventing a backlog of outdated frames from accumulating and introducing delays. **On the processing side** by choosing a lightweight detection model keeps inference time low. **On the output side** using the native Android TTS API avoids any overhead associated with remote audio generation or network transmission, since the entire speech synthesis process happens locally on the device.

Through the combined effect of all these optimizations the system achieves an end-to-end latency within the one-second threshold required for the audio descriptions to accurately reflect the user’s current environment at all times.

5 Conclusion

This chapter presented the main techniques behind the proposed assistive system. Starting from the acquisition and preprocessing of raw image data to detecting and localizing multiple objects within a scene in a matter of milliseconds. Examining after that how the geometric output of the detection is transformed into spatial information through bounding box analysis and position estimation and finally, analyzing how the structured detection results are converted into natural language descriptions and delivered as spoken audio through a TTS engine .

With the theoretical and technical foundations now established, the next chapter focuses on the system design and implementation of a functional application.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

Sommaire

1	Introduction	25
2	Requirements analysis	25
2.1	Target user profile	25
2.2	Use cases	26
2.3	Mobility scenarios	28
3	System specifications	29
3.1	Functional requirements	29
3.2	Non-functional requirements	29
4	System architecture	30
5	UML modeling	31
5.1	Use case diagram	31
5.2	Class diagram	32
5.3	Sequence diagram	33
6	Conclusion	35

1 Introduction

Based upon the theoretical foundations in the previous chapters, this chapter translates the identified user needs and technical approaches into a formal system design. It covers the requirements analysis, functional and non-functional specifications, system architecture and UML modeling of the application.

2 Requirements analysis

Before any system can be designed, it must be understood as a human problem before a technical one. This section identifies who will use the application, what they need from it and the concrete situations it must handle.

2.1 Target user profile

The primary target users of this application are blind or severely visually impaired individuals. These users have a common set of characteristics that directly shape every design decision made in this project.

From an **interaction perspective**: Any interface element that requires visual engagement is unusable for this population. This constraint shaped one of the key design decisions of this project. Rather than requiring the user to locate and tap a button on screen to start or stop the detection process, the application uses the physical volume button of the smartphone which a blind user can locate only by touch. This what makes the interaction completely eyes-free and accessible from the very first use without any prior configuration or learning.

From a **usage perspective**, blind users need assistive tools that work during active mobility meaning all interaction must be hands-free, all output must be delivered through audio output and response time must be low enough to follow the user's surroundings as it changes in RT.

2.2 Use cases

The following use cases define the interactions between the user and the system.

2.2.1 Use case 1: Start RT detection

This use case is the entry point of the assistive system. The user triggers the entire detection pipeline by pressing the physical volume button . It's the most frequently used feature of the application and everything else depends on it being successfully activated.

Table 3.1: Use case description: Start RT Detection

Title	Start RT detection
Summary	The user activates the system to begin RT object detection and receive audio guidance.
Actor	Blind User
Pre-condition	The application is installed, permissions are granted and the system is ready.
Post-condition	The detection process is active and the system continuously analyzes the environment.
Nominal scenario	1. The user presses the physical volume button. 2. The system activates the camera. 3. The detection process starts. 4. The system begins analyzing the environment in RT.
Alternative scenario	A1. Camera permission not granted: the system requests permission or announces an error. A2. System not ready: the system informs the user that detection cannot start.

2.2.2 Use case 2: Receive audio guidance

This use case represents the output delivered by the system. It occurs automatically and repeatedly as long as detection is active requiring no user action and assures the transforming of visual information into spoken guidance.

Table 3.2: Use case description: Receive audio guidance

Title	Receive audio guidance
Summary	The system detects objects in the environment and provides spoken descriptions to the user.
Actor	System
Pre-condition	The detection process is active.
Post-condition	The user receives an audio description of detected objects and their positions.
Nominal scenario	1. The system captures a frame from the camera. 2. Objects in the scene are detected. 3. The system determines their positions (left, center, right). 4. A natural language description is generated. 5. The description is spoken to the user.
Alternative scenario	A1. No relevant objects detected: no message is spoken. A2. Repeated detection: the system suppresses duplicate announcements.

2.2.3 Use case 3: Priority alert for critical objects

This use case ensures that safety-critical information is delivered firstly to user. The system interrupts the output to prioritize what matters most directly reducing the risk of the user missing a dangerous obstacle.

Table 3.3: Use case description: Priority alert for critical objects

Title	Priority alert for critical objects
Summary	The system prioritizes and alerts the user about critical objects that may pose a danger.
Actor	System
Pre-condition	Detection is active and objects are detected.
Post-condition	A prioritized warning is delivered to the user.
Nominal scenario	1. The system detects multiple objects. 2. Critical objects (obstacles, vehicles) are identified. 3. The system prioritizes these objects. 4. A warning message is generated. 5. The message is spoken immediately.
Alternative scenario	A1. No critical objects: normal descriptions are provided. A2. Multiple critical objects: the closest one is prioritized.

2.2.4 Use case 4: Stop detection

This use case gives the user full control over when the system is active or not . The same physical volume button used to start detection is pressed again to stop it.

Table 3.4: Use case description: Stop detection

Title	Stop detection
Summary	The user deactivates the detection mode to silence the audio output when assistive guidance is no longer needed.
Actor	Blind user
Pre-condition	The detection loop is currently active.
Post-condition	The detection process is stopped, the camera is deactivated and any ongoing TTS speech is interrupted.
Nominal scenario	1. The user presses the physical volume button again. 2. The detection loop is stopped. 3. Any ongoing TTS speech is immediately interrupted. 4. The camera deactivates.
Alternative scenario	A1. The application is closed by the user: the detection loop and TTS are automatically stopped.

2.3 Mobility scenarios

The use cases defined above showed what the system must do. The following scenarios define the real world contexts in which it must perform reliably.

The application is designed to support three main navigation scenarios.

The first scenario covers **indoor navigation** whether at home or in public spaces. At home, the user moves through familiar domestic spaces where the primary detection needs are furniture obstacles and door detection under generally stable but sometimes poor lighting. In public spaces the environment becomes more dynamic with moving people and unfamiliar layouts making people detection and obstacle avoidance the highest priority.

The second scenario is **Outdoor navigation** that present the most demanding conditions by requiring accurate detection of moving objects and peoples across highly variable lighting including direct sunlight, shadows or nighttime conditions.

The third scenario is **Public transport use** where the user boards or exits buses-trams or metros. In this context detecting vehicle doors, steps and surrounding passengers becomes essential.

3 System specifications

This section defines the requirements the system must satisfy to be considered functional, reliable and accessible in real-world assistive use based on the user needs and technical analysis. The system must satisfy the following requirements:

3.1 Functional requirements

The system should be able to detect objects in a matter of seconds and announce their position and distance via spoken audio without any user interaction. Safety critical objects must always be communicated first and repeated announcements for unchanged detections must be suppressed. The entire interaction must be controllable through a volume button alone requiring no visual engagement and no configuration from the user.

3.2 Non-functional requirements

The system should maintain an end-to-end latency below one second from frame capture to audio output across extended usage sessions. It must detect reliably under variable lighting conditions and handle failure scenarios such as network interruptions or server unavailability through spoken error messages rather than silent crashes. The audio output have to be clear and configurable in terms of speech rate and language to assure the diverse needs of the target user population.

These specifications together build the behavioral and quality boundaries that the system have to satisfy. The following section translates these requirements into an architectural design that defines how each component of the system is structured.

4 System architecture

The application is built around a fully on-device processing pipeline that connects three sequential stages: image capture, AI-based object detection and spoken audio output. The entire process runs locally on the phone with no dependency on any external server or network connection. The figure 3.1 illustrates the entire process

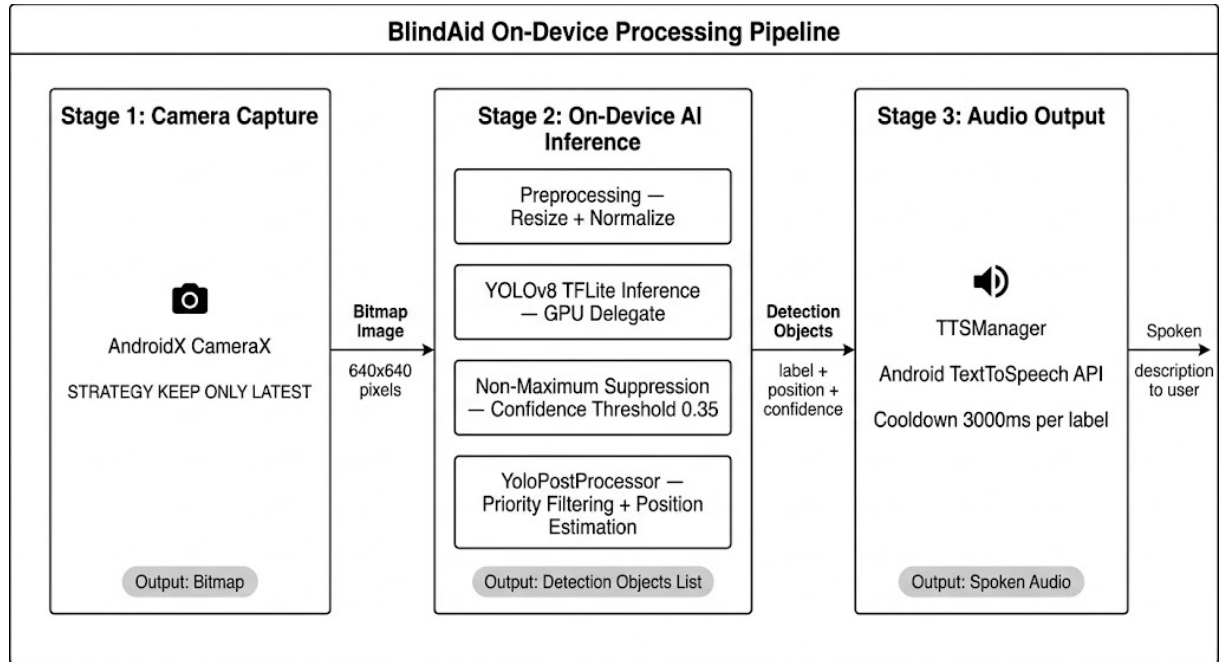


Figure 3.1: On-device processing pipeline

The pipeline begins with the camera stage. The phone’s camera continuously captures frames from the smartphone camera using a strategy that ensures that only the most recent frame enters the inference pipeline while older frames are automatically discarded keeping the audio description synchronized with the user’s current surroundings. Each captured frame is converted to a rotated Bitmap to match the device orientation then passed immediately to the inference stage.

The inference stage is handled by the TFLiteDetector component. Each incoming Bitmap is resized to 640×640 pixels and normalized before passing to the fine-tuned YOLOv8 model running through the TensorFlow Lite runtime. The raw output tensor is then processed by the YoloPostProcessor which applies a confidence threshold filter, performs NMS to remove duplicate bounding boxes, computes the spatial position of each detection, applies priority filtering and assembles a natural language description string. The final

stage is the audio output. The assembled description string is passed to the TTSTManager which delivers it as spoken audio through the native Android TTS API. The entire pipeline from frame capture to spoken output achieves an end-to-end latency of approximately one to two seconds, confirmed during real-world testing on an entry-level Android device.

5 UML modeling

This section provides a Unified Modeling Language (UML) representation of the system through three complementary diagrams: **a use case diagram, a class diagram and sequence diagrams** capturing a different dimension of the application's structure.

5.1 Use case diagram

The use case diagram presents the interactions between the blind user and the system. The four identified use cases are: **Start detection, Receive audio guidance, Priority alert for critical objects and Stop detection.**

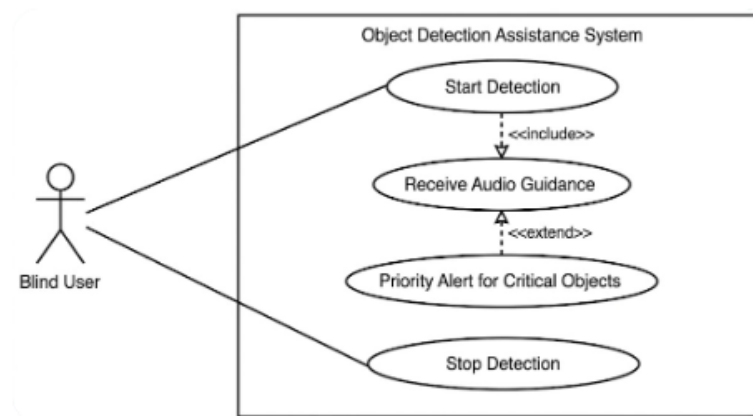


Figure 3.2: Use case diagram for the object detection system established using StarUML

As illustrated in Figure 3.2, only the use cases Start detection and Stop detection are triggered by the user due to the eye-free experience this application rely on. Receive audio guidance is "included" by Start detection as it occurs with every detection cycle while Priorityalert "extends" it conditionally when safety-critical objects are identified in the scene.

5.2 Class diagram

The class diagram 3.3 presents the static structure of the **BlindAid** application by showing the main classes, their attributes, methods and relationships. The **FrameAnalyzer** acts as the central coordinator of the pipeline by managing the two volatile flags **detectionActive** and **detectorReady** to ensure thread-safe operation across the camera, inference and audio threads. It uses the four following main service classes. The **CameraManager** handles all camera operations through the AndroidX CameraX by capturing frames and converting them to Bitmap format ready for inference. The **TFLiteDetector** is responsible for loading the TFLite model at startup, preprocessing each incoming Bitmap through resizing and normalization and running YOLOv8 inference. The **YoloPostProcessor** interprets the raw output tensor by applying confidence filtering, NMS, spatial position estimation and priority filtering before assembling the final natural language description string. The **TTSManager** manages all spoken output through the native Android TTS and enforces the per-label cooldown mechanism to suppress repeated announcements. Finally, **Detection** and **BoundingBox** are value objects produced at each detection cycle carrying the class label, confidence score, spatial position, distance estimate and raw bounding box coordinates used for spatial reasoning.

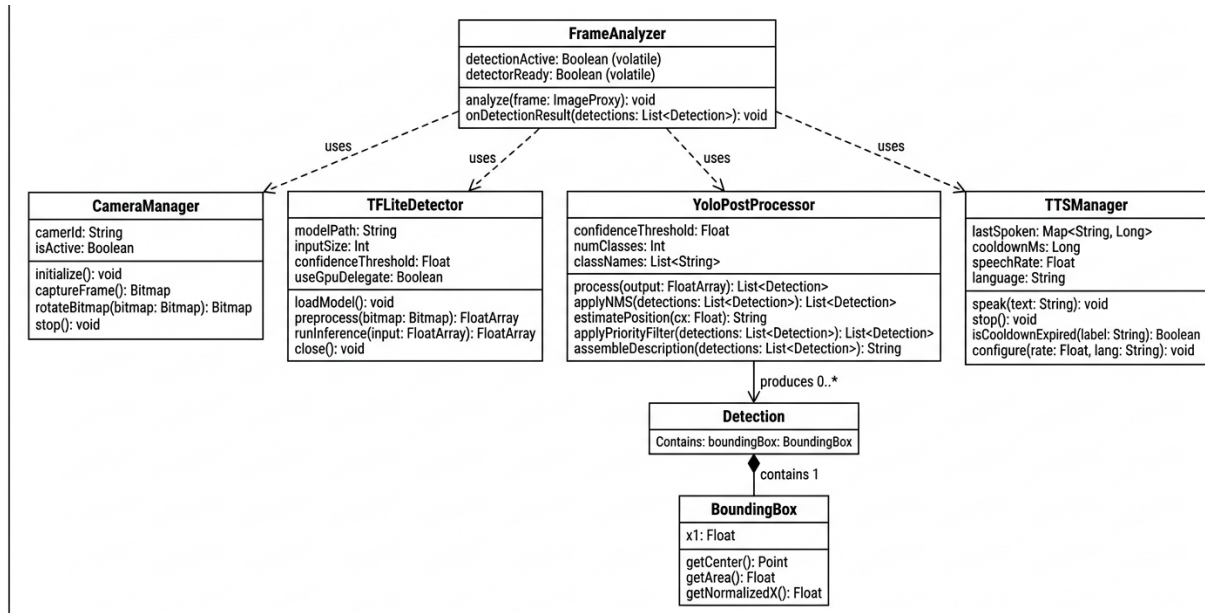


Figure 3.3: Class diagram of the Object Detection Assistance System using StarUML

5.3 Sequence diagram

Three sequence diagrams illustrate the dynamic behavior of the system.

5.3.1 Sequence diagram 1 — Start Detection

The diagram 3.4 describes the sequence triggered when the user presses the volume button to activate the system. The application initializes the FrameAnalyzer which coordinates the CameraManager to load current CameraX configurations. Simultaneousl, the TFLiteDetector is initialized to load the YOLOv8 model into memory. Rather than relying on a remote server, the FrameAnalyzer starts a local detection loop where frames are captured and processed entirely on-device. The detections results are then processed by the TTSMManager, which transfers the generated descriptions to the audio output for user feedback.”

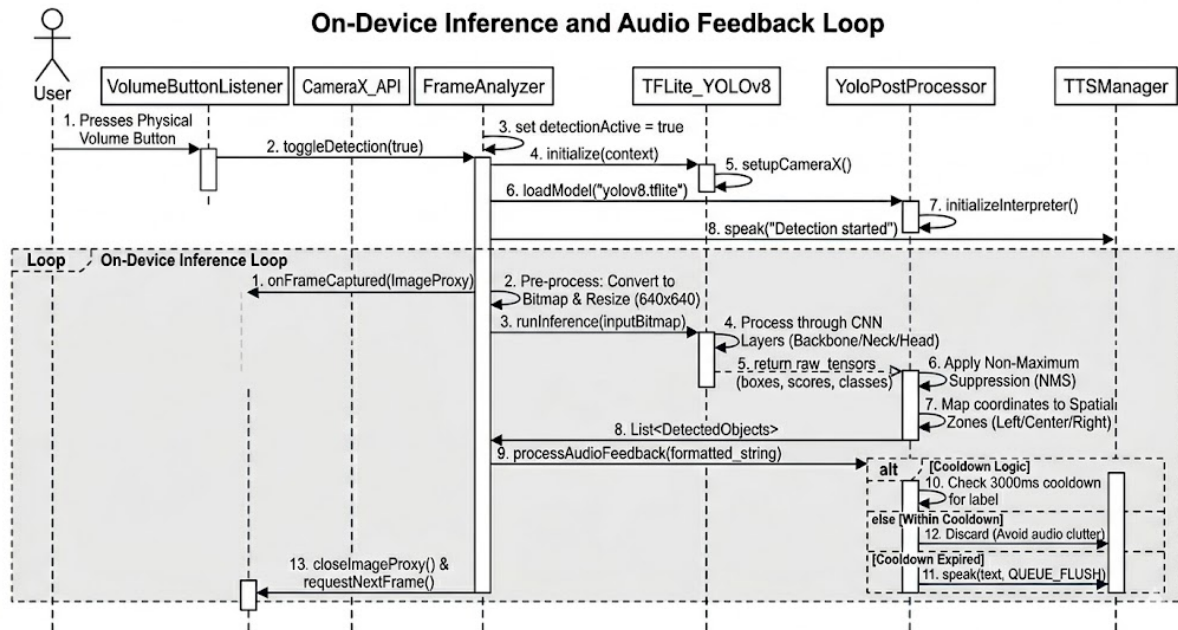


Figure 3.4: Sequence diagram: Start detection

5.3.2 Sequence diagram 2 — Receive Audio Description

The following 3.5 diagram focuses on the continuous loop of the system as it provides audio guidance in RT. On each frame update, FrameAnalyzer performs local pre-processing by resizing the image to the resolution required by the model then passed to the the YOLOv8

model. The resulting raw data is handled by the YoloPostProcessor which filters the results using NMS and maps the coordinates to spatial zones such as "Left" or "Right." Finally, the TTS engine receives the description to ensure the audio feedback is clear and not repetitive, providing the user with timely and organized information.

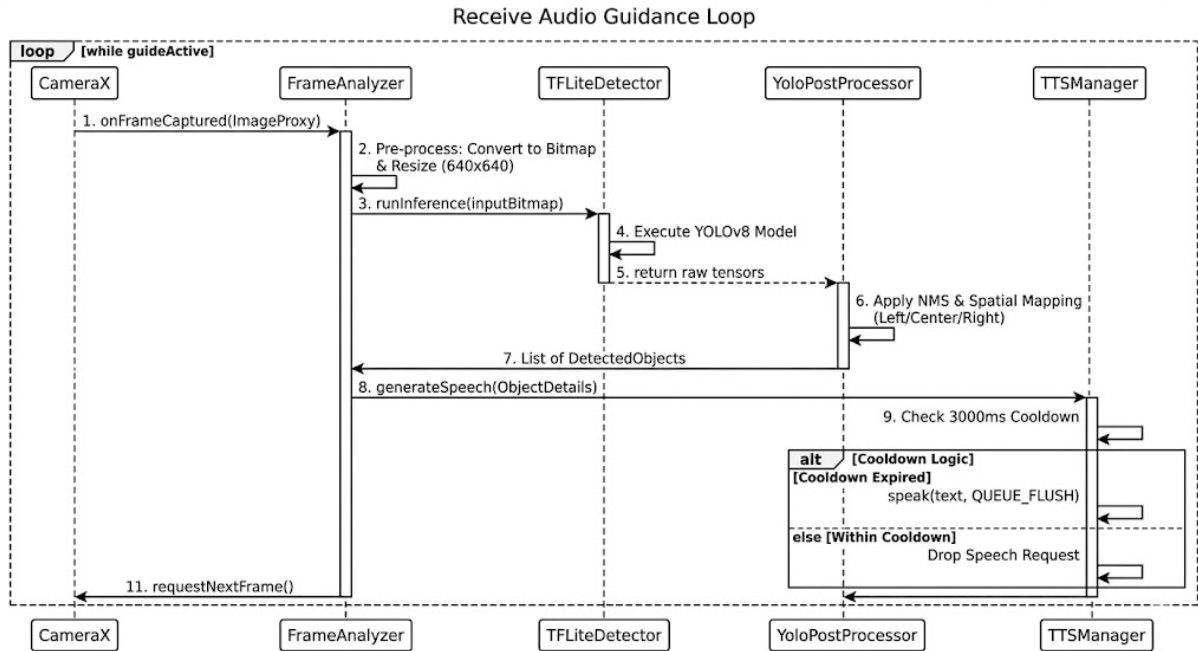


Figure 3.5: Sequence diagram: Receive audio description

5.3.3 Sequence diagram 3 — Stop detection

This diagram 3.6 illustrates the sequence triggered to finish the scanning process. Upon a secondary interaction with the physical volume button, a deactivation request is sent to the FrameAnalyzer which immediately switches the system's operational status to inactive. This state change prompts the CameraManager to stop all active CameraX use cases and powering down the camera hardware to conserve battery. Simultaneously, the FrameAnalyzer signals the TTSTManager to stop any ongoing auditory descriptions and clear the speech queue. The sequence concludes with a final vocal confirmation "detection stopped" to the user.

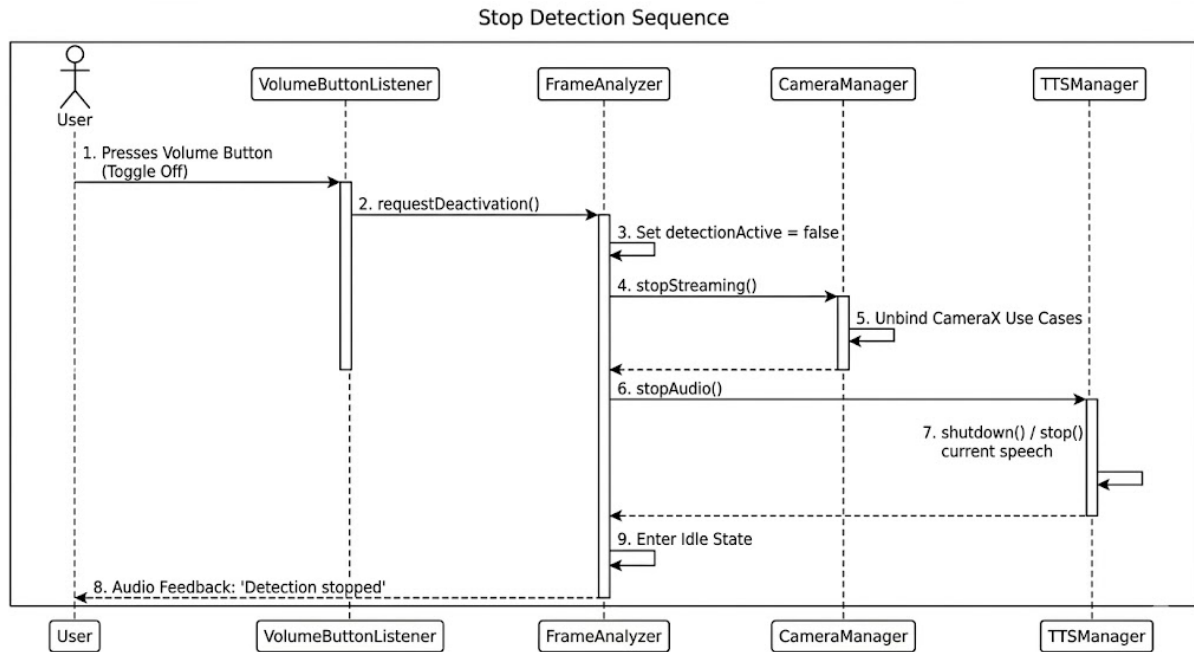


Figure 3.6: Sequence diagram: Stop detection

6 Conclusion

This chapter presented the complete design of the application by translating the user needs and techniques identified in the previous chapters into a structured system specifications. The analysis formed a precise understanding of the target user population and the scenarios the system must support. The functional and non-functional specifications defined the behavioral and quality requirements that the implementation must satisfy. The UML modeling provided the representation of the system through a use case diagram, a class diagram and a three sequence diagrams covering the important interaction scenarios.

The following chapter will describe how this design was concretely realized, tested and evaluated.

CHAPTER 4

IMPLEMENTATION, TESTING AND EVALUATION

1 Introduction

This chapter presents the concrete realization of the system designed in the previous chapters. It covers the development environment, the implementation of each pipeline component and the testing used to validate the application.

2 Development environment

2.1 Mobile platform

The application **BlindAid** was developed using Android Studio Panda as the primary IDE. The application was written in Kotlin which is the officially recommended language for Android development that targets Android API level 24 (Android 7.0) as the minimum SDK and API level 35 as the target SDK.

The deploying and testing took place on an Infinix smartphone running Android 10 Go Edition with the XOS interface that is designed for entry-level hardware with limited RAM and processing power. This choice validates one of the objectives which is that **BlindAid must function on affordable devices accessible to a wider population**.

Table 4.1 summarizes the mobile development environment.

Table 4.1: Mobile Development Environment

Component	Details
IDE	Android Studio Panda
Language	Kotlin
Min SDK	API 24 (Android 7.0)
Target SDK	API 35
Test Device	Infinix XOS
Operating System	Android 10 Go Edition
Camera Library	AndroidX CameraX 1.3.1
Inference Engine	TensorFlow Lite 2.14.0
TTS Engine	Android Native TextToSpeech API
Build System	Gradle 8.7 with Kotlin DSL
AGP Version	8.5.2

2.2 AI frameworks

The object detection component of **BlindAid** relies on a YOLOv8 model that was fine-tuned on Google Colab using the Ultralytics library and PyTorch, then converted to TensorFlow Lite (TFLite) format for deployment on mobile devices.



Figure 4.1: TensorFlow Lite framework used for on-device inference (source: [?])

TensorFlow Lite is a lightweight deep learning framework designed for on-device inference, enabling low-latency execution and reduced resource consumption. Unlike server-based

approaches that require network connectivity and backend infrastructure, TFLite allows **BlindAid** to run the entire inference pipeline locally on the device, ensuring full offline functionality and improved responsiveness.

The fine-tuned model was exported in multiple quantization formats. The float16 variant was selected for deployment as it offers a reduced model size and faster inference compared to float32, while preserving higher accuracy than int8 quantization. Additionally, it is compatible with the Android GPU delegate, enabling hardware-accelerated inference when supported, with automatic fallback to CPU execution on unsupported devices.

Table 4.2 summarizes the AI development environment.

Table 4.2: AI Development Environment

Component	Details
Base model	YOLOv8 (Ultralytics)
Training platform	Google Colab (Tesla T4 GPU)
Framework	Ultralytics 8.4.41 / PyTorch 2.1.0
Inference engine	TensorFlow Lite 2.14.0
Hardware acceleration	GPU Delegate with CPU fallback
Model format	Float16 TFLite
Dataset management	Roboflow

2.3 Testing tools

System testing was conducted entirely on an Infinix device without any external server which reflects the real deployment conditions of the application. Testing accumulated over 10 hours of usage across multiple environments . Additional informal testing was conducted with a group of friends who interacted with the application without prior instruction, providing qualitative feedback on the clarity and usefulness of the audio descriptions

3 Implementation of the Detection Module

3.1 Overview

BlindAid is structured around 5 Kotlin components that form a sequential processing pipeline.



Figure 4.2: BlindAid’s processing pipeline.

This architecture makes sure that every stage can be independently maintained or replaced. For example, the detection model can be updated by replacing the **model.tflite** file without modifying any Kotlin source code since class names are loaded dynamically from a different companion.

3.2 Camera integration

The camera system is implemented using the AndroidX CameraX library that is configured with the **STRATEGY_KEEP_ONLY_LATEST** strategy because it ensures that when inference takes longer than frame delivery, the older frames are automatically discarded and only the most recent frame is processed, while keeping the audio description synchronized with the user’s current scene.

Furthermore , the camera runs on a different background thread which isolates frame processing from the main UI thread and prevents application from freezing .

Each captured frame is converted to a Bitmap, rotated to match the device orientation and passed to the **TFLiteDetector** for preprocessing and inference.

3.3 Model loading and fine-Tuning

3.3.1 Dataset preparation

The YOLOv8 model was not deployed using standard COCO pretrained weights alone. A fine-tuning process was conducted on Google Colab to improve detection performance on important object categories .

Dataset preparation was performed on Roboflow, where a COCO-based dataset of 10,163 annotated images were assembled and configured for YOLOv8 training. Figure 4.3 shows the dataset composition and configuration on the Roboflow platform.

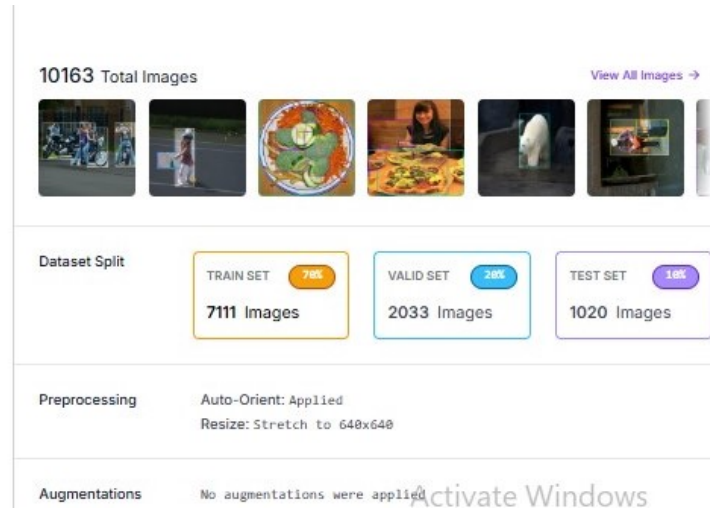
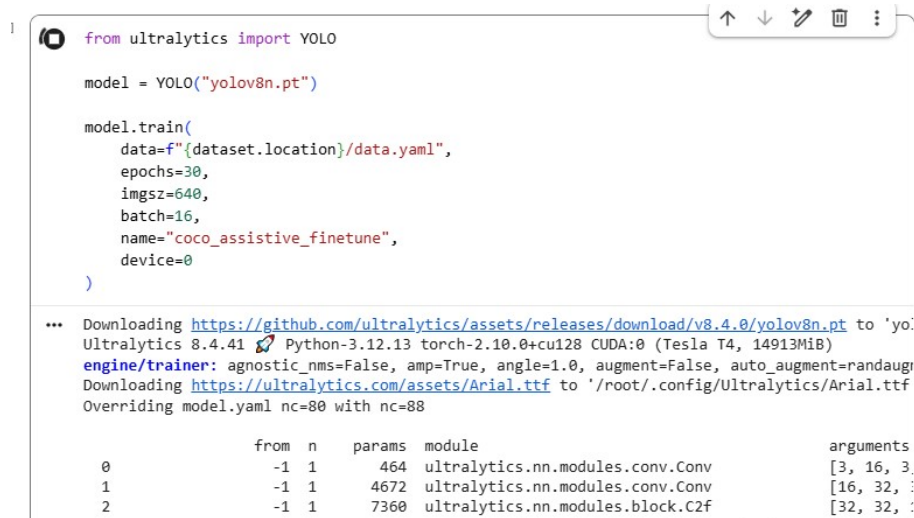


Figure 4.3: Roboflow dataset configuration

The dataset was split into 7,111 training images (70%), 2,033 validation images (20%) and 1,020 test images (10%). An automatic orientation correction and resizing to 640*640 pixels during preprocessing was applied . The dataset covers 88 object classes, extending the standard 80-class COCO set with additional categories .

3.3.2 Training configuration

The fine-tuning process was executed on Google Colab using a Tesla T4 GPU with 14,913 MiB of VRAM, running Ultralytics 8.4.41 with PyTorch 2.1.0 and CUDA acceleration. picture 4.4 presents the training configuration:



```

1 from ultralytics import YOLO

model = YOLO("yolov8n.pt")

model.train(
    data=f"{dataset.location}/data.yaml",
    epochs=30,
    imgsz=640,
    batch=16,
    name="coco_assistive_finetune",
    device=0
)

... Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yo
Ultralytics 8.4.41 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugr
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf'
Overriding model.yaml nc=80 with nc=88

      from  n  params  module  arguments
0         -1  1      464  ultralytics.nn.modules.conv.Conv  [3, 16, 3
1         -1  1     4672  ultralytics.nn.modules.conv.Conv  [16, 32, 3
2         -1  1     7360  ultralytics.nn.modules.block.C2f  [32, 32, 3

```

Figure 4.4: YOLOv8 Fine-Tuning configuration on Google Collab

3.3.3 Fine-Tuning results

The fine-tuned model was evaluated on the held-out test set of 1,020 images. Table 4.3 summarizes the performance metrics obtained after 30 training epochs.

Table 4.3: YOLOv8 Fine-Tuning performance Metrics

Metric	Value	Description
Precision	57.16%	Proportion of correct positive detections
Recall	44.72%	Proportion of actual objects detected
mAP50	46.52%	Mean Average Precision at IoU 0.50
mAP50-95	31.47%	Mean AP averaged over IoU 0.50–0.95

Figure 4.5 visualizes these metrics across the four evaluation criteria:

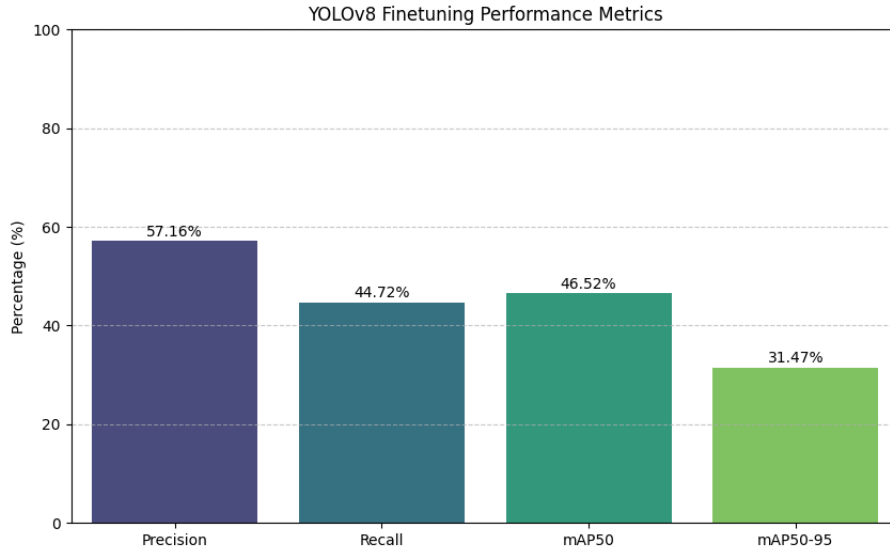


Figure 4.5: YOLOv8 fine-tuning performance metrics .

The **mAP50** of 46.52% represents an improvement over the baseline model which achieved only 8.97% at epoch 1 which confirms that fine-tuning effectively adapted the model to the target object distribution. The **precision** of 57.16% indicates reliable detection quality. The **recall** of 44.72% reflects the inherent challenge of covering 88 classes with a lightweight nano-scale model and identifies a clear direction for future improvement.

3.4 TFLite inference engine

The fine-tuned model was exported to TensorFlow Lite format and deployed as **model.tflite**.

The inference engine is configured with a GPU delegate as the primary execution option with automatic fallback to CPU using 4 threads if the GPU delegate is unavailable on the device. This ensures compatibility across a wide range of Android hardware configurations.

Each input frame is resized to 640*640 pixels and normalized to the range [0.0,- 1.0] before being passed to the model. The output tensor has shape **[1, 4+numClasses, 8400]**, where 8400 corresponds to the number of anchor predictions generated for a 640*640 input. The first 4 values encode the bounding box coordinates and the other values contain the confidence score per class.

3.5 RT post-processing

The raw model output is processed by the **YoloPostProcessor** component. Detections below a confidence threshold of 0.35 are discarded and NMS is applied to remove duplicate bounding boxes. The remaining detection results with the highest confidence score are for announcement.

Position estimation is then computed from the normalized horizontal center of the bounding box as described in Table 4.4.

Table 4.4: Position Estimation Logic

Condition	Announced Position
$cx < 0.33$	"on your left"
$cx > 0.66$	"on your right"
$0.33 \leq cx \leq 0.66$	"ahead of you"

4 Implementation of the Vocal Description Module

4.1 Text generation

Once the detected object label is combined with its position the next step is to form a natural language phrase. The description is assembled on the device using a rule-based template system with each detection tuple containing a class label and a spatial zone. This rule-based approach guarantees deterministic and low-latency output **with no neural language model involved at this stage**.

4.2 Speech synthesis

Audio output is handled by the Android native **TextToSpeech API**. The engine operates entirely on-device ensuring that audio feedback remains available regardless of connectivity conditions. All announcements use **QUEUE_FLUSH** mode meaning each new announcement interrupts any speech currently playing ensuring that current environmental information is always prioritized.

4.3 Audio queue management

A per-label cooldown of 3000ms is enforced to prevent repetitive audio when the same object remains in the scene across multiple detection cycles. Each class label is mapped to its last announcement timestamp and if the cooldown has not expired the announcement is suppressed. System messages such as "Detection started" bypass the cooldown and are always spoken immediately when a user launches the app.

5 Accessible User Interface

5.1 Simplified interface

BlindAid was designed to be fully usable without any visual engagement. As shown in Figure 4.6, the application logo reflects its core mission: the assistive vision is represented by an eye held in a supporting hand.



Figure 4.6: BlindAid logo.



Figure 4.7: BlindAid main interface detecting Bsbs with 84% accuracy .

The main interface is a minimal layout showing the current detection label, confidence score and spatial position. The camera operates silently in the background feeding directly into the inference pipeline along with the audio-first design of the application.

The TTS engine announces firstly that the app is ready, confirming that **BlindAid** requires is eye-free at any point during use. The real-world detection example shown in Figure 4.7 illustrates the system output format: the detected class, confidence percentage and spatial position are displayed on screen while simultaneously spoken to the user.

5.2 Volume button control

The primary interaction mechanism is the physical volume button of the phone . Pressing the volume key toggles the start and the end of . When toggled on, the system announces "Detection started" and begins processing frames. When toggled off, it announces "Detection stopped". This physical input can be located by touch without any visual interaction guaranteeing the most accessible control for blind and visually impaired users.

5.3 Threading and lifecycle management

BlindAid uses three distinct threads to ensure stable and responsive operation. The **main UI thread** handles layout updates, volume key operations and permissions. A dedicated **background thread** initializes the TFLiteDetector on app start to prevent freezes during model loading. A **camera executor thread** to handle frame analysis, inference and post-processing. The `detectionActive` and `detectorReady` flags are marked as volatile to ensure safe reading across all threads.

6 Testing and evaluation

6.1 RT performance

Since **BlindAid** runs entirely on-device with no network dependency, end-to-end latency is determined solely by on-device inference and TTS response time.

Real-world testing showed an end-to-end latency of approximately 1–2 seconds from frame capture to audio output. This delay is mainly due to preprocessing (50–100 ms), CPU-based TFLite inference (800–1400 ms), and TTS synthesis (100–200 ms), resulting in an effective detection rate of about 1 frame per second, potentially reaching 2–5 FPS with hardware acceleration. Despite hardware limitations, the system meets real-time requirements, with performance expected to significantly improve on more capable devices.

6.2 Real-world testing

BlindAid was tested across multiple environments by the developer and a group of friends, accumulating over 10 hours of active usage covering indoor spaces and outdoor environments.

Indoor testing under stable lighting yielded the most consistent results, with common objects and persons reliably detected and localized. Outdoor testing showed acceptable performance for high-priority objects, although detection confidence dropped under strong direct sunlight.

Friend testers with no prior familiarity with the application were able to understand and act on the audio descriptions without instruction, validating the clarity of the spoken output.

6.3 Limitations

Despite the encouraging results, **BlindAid** presents several limitations in its current form.

The model recall of 44.72% means that a proportion of objects in the scene are not detected and therefore not announced. While this is acceptable for low-priority items, missed detections of stairs, curbs, or moving vehicles may represent potential safety risks.

Another limitation is that the current implementation announces only the single highest-confidence detection per cycle. In complex scenes with multiple simultaneous obstacles, this may result in important objects being omitted in a given announcement cycle.

6.4 Future improvements

Several directions have been identified for future development of **BlindAid**.

Extending the training dataset with images captured specifically in assistive navigation contexts would directly improve performance on object categories most critical to user safety. In particular, building and incorporating a dataset collected in Tunisian environments would enhance the model’s robustness and relevance to the target deployment context.

Migrating to a larger model variant such as YOLOv8s with additional fine-tuning epochs could further improve detection accuracy while remaining within acceptable latency bounds on mid-range devices.

Announcing multiple detected objects per cycle rather than only the top detection would provide richer situational awareness, especially in crowded environments.

Finally, adding distance estimation based on bounding box area would allow the system to convey urgency, for example distinguishing between nearby and distant objects.

7 Conclusion

This chapter translated the system design of the previous chapters into a fully functional Android application. Each component of the pipeline from camera capture to spatial reasoning and spoken output was implemented, integrated and validated through real-world testing. The results confirmed that **BlindAid** meets its core objective of delivering reliable, RT audio guidance to blind and visually impaired users on an affordable mobile hardware. The limitations identified open clear directions for future development, which are addressed in the general conclusion of this report.

GENERAL CONCLUSION AND PERSPECTIVES

This project presented the design, development and evaluation of **BlindAid** :an AI-based mobile application aimed at improving the safety of blind and visually impaired individuals. Driven by the observation that blind people worldwide live with some form of vision impairment and that existing assistive tools remain limited in contextual information and out of reach of most people , this work set out to build an intelligent, affordable and fully offline alternative.

The system was built around a complete on-device architecture : a smartphone camera continuously captures the user's surroundings ,a fine-tuned YOLOv8 model running through TensorFlow Lite detects and localizes objects in RT, a rule-based natural language module assembles spatial descriptions and the Android native TTS engine delivers them as spoken audio. All processing runs locally on the device with no dependency on network connectivity or external infrastructure, making the application reliable in any context.

The fine-tuning of YOLOv8 was on a custom COCO-based dataset of over 10,000 annotated images yielded meaningful improvements raising the precision and accuracy of the model . Real-world testing across indoor and outdoor environments took over 10 hours of usage confirmed that the **BlindAid** consistently delivers audio feedback within one to two seconds of frame capture, even on an low-level Android devices.In addition,users with

no prior experience were able to understand and act on the spoken descriptions without any instruction which validates the clarity and accessibility of the audio output.

Despite these encouraging results, the project identified several limitations. The recall percentage obtained means that a portion of objects in complex scenes remain unannounced which can represent a safety risk for critical categories. The current implementation also limits each announcement cycle to the single highest-confidence detection reducing environmental awareness in crowded scenes.

Several directions are envisioned for future development. Expanding the training dataset with images collected specifically in assistive navigation contexts would directly improve detection coverage. Migrating to a larger model variant with additional fine-tuning epochs could raise recall without exceeding acceptable latency bounds. Announcing multiple detected objects per cycle and integrating depth-based distance estimation would provide richer and more actionable guidance. On a longer-term, incorporating natural language models to generate more descriptive and context-aware audio feedback as well as conducting formal usability studies with actual blind users would bring **BlindAid** closer to a production-ready assistive tool.

Ultimately, BlindAid stands as proof that AI, when thoughtfully applied to human needs can bridge the gap between disability and independence by transforming a regular smartphone into a reliable pair of eyes that not only detects obstacles but understands them, locates them and communicates them in a way that empowers the user to navigate the world with greater confidence and safety.

BIBLIOGRAPHY

- [1] World Health Organization, *World Report on Vision*, Geneva: World Health Organization, 2019. Available at: <https://www.who.int/publications/i/item/world-report-on-vision/> Accessed: April 2025.
- [2] OrCam Technologies Ltd., *OrCam MyEye* [Assistive Device], 2023. <https://www.orcam.com/en/myeye/>. Accessed: April 2025.
- [3] Be My Eyes, *Be My Eyes* [Mobile Application], 2015. <https://www.bemyeyes.com>. Accessed: April 2025.
- [4] Aira Tech Corp., *Aira: Visual Interpreting Service* [Online Service], 2017. <https://aira.io>. Accessed: April 2025.
- [5] G. Jocher, A. Chaurasia and J. Qiu, *Ultralytics YOLOv8*, version 8.0.0, 2023. <https://github.com/ultralytics/ultralytics>.
- [6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 740–755.
- [7] S. Ren, K. He, R. Girshick and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 91–99.

- [8] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [9] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [10] Roboflow Inc., *Roboflow* [Computer software], 2024. <https://roboflow.com>.
- [11] Pallets Projects, *Flask: A Lightweight WSGI Web Application Framework*, 2010. <https://flask.palletsprojects.com>.
- [12] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [13] Google LLC, *Android CameraX API*, Android Developers Documentation, 2024. <https://developer.android.com/training/camerax>.
- [14] Google LLC, *Android TextToSpeech API*, Android Developers Documentation, 2024. <https://developer.android.com/reference/android/speech/tts/TextToSpeech>.
- [15] Google LLC, *Google Colaboratory* [Online platform], 2017. <https://colab.research.google.com>. Accessed: April 2025.